CS 301
High-Performance Computing

Assignment 04

Particle Interpolation and Mover

Jas Mehta (202301432)
Yug Tejani (202301487)

March 10, 2026

Contents

1 Introduction and Problem Description **3**

2 Hardware Details **3**
2.1 Hardware Details for Lab PC 3
2.2 Hardware Details for HPC Cluster (Node gics2) 3

3 Methodology and Overall Approach **4**

4 Graphical Results **4**
4.1 Experiment 01: Scaling with Number of Particles 4
4.2 Experiment 02: Consistency Across Configurations 6
4.3 Experiment 03: The Mover Operation and OpenMP Parallelization 7
4.3.1 Lab PC Results 7
4.3.2 HPC Cluster Results 8

5 Analysis and Questions **10**
5.1 Performance Differences: Lab PC vs. HPC Cluster 10
5.2 Parallel Speedup and Hardware Properties 10
5.3 Parallelizing the Interpolation Function 10

1 Introduction and Problem Description

In this assignment, we extend the scattered point to mesh interpolation scheme by analyzing its performance under various conditions and introducing a particle tracking phase. All particles and grid points lie within a specific domain:

$$1 \times 1$$

We conduct three specific experiments: exploring the scaling of execution time with particle count, checking the consistency of interpolation time across iterations, and introducing a particle Mover phase which is subsequently parallelized using OpenMP.

2 Hardware Details

2.1 Hardware Details for Lab PC

- Architecture: x86_64
- CPU(s): 12
- Thread(s) per core: 2
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- L3 cache: 18M

2.2 Hardware Details for HPC Cluster (Node gics2)

- Architecture: x86_64
- CPU(s): 16
- Thread(s) per core: 1
- Core(s) per socket: 8
- Socket(s): 2
- NUMA node(s): 2
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
- L3 cache: 20480K

3 Methodology and Overall Approach

The code initializes a large array of particle coordinates and a mesh grid. The interpolation scheme iterates over the particles, calculating their relative weights, and distributes those weights to the four nearest grid nodes. The mover scheme then updates the coordinates of the particles.

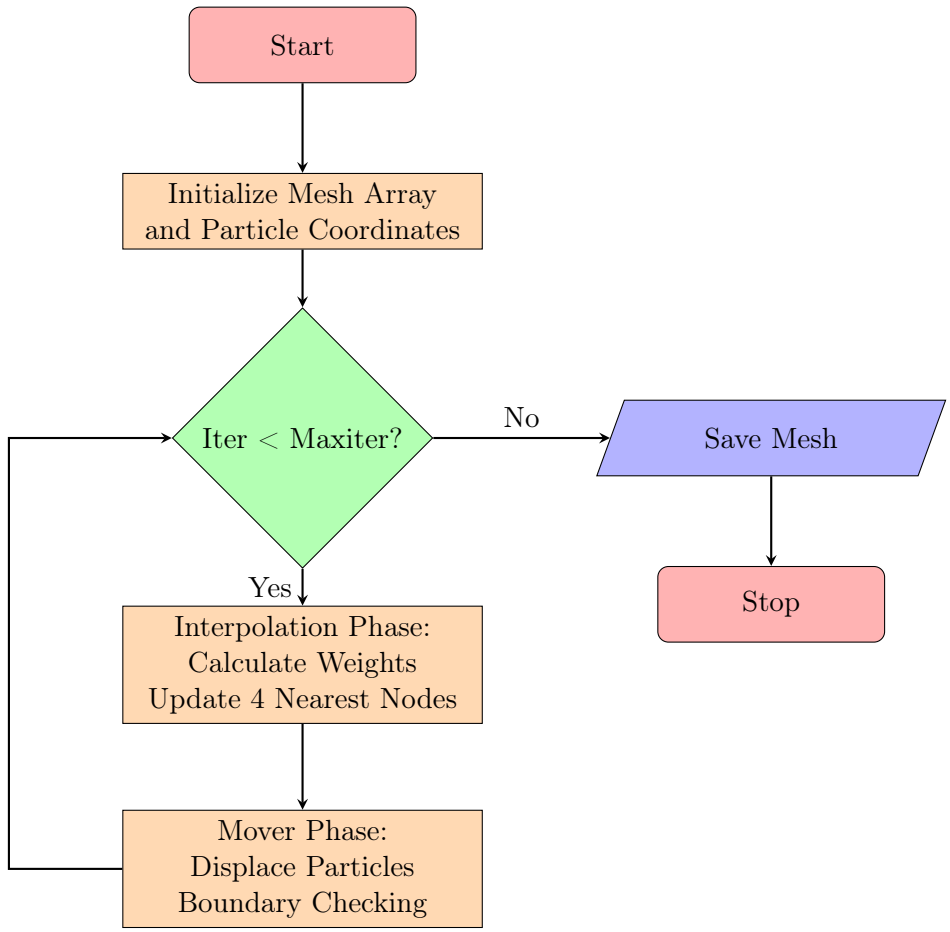

Figure 1: Flow diagram of the Interpolation and Mover algorithm.

4 Graphical Results

4.1 Experiment 01: Scaling with Number of Particles

In this experiment, we observe how the interpolation execution time scales with the number of particles. The particle counts tested are:

$$\{10^2, 10^4, 10^6, 10^8, 10^9\}$$

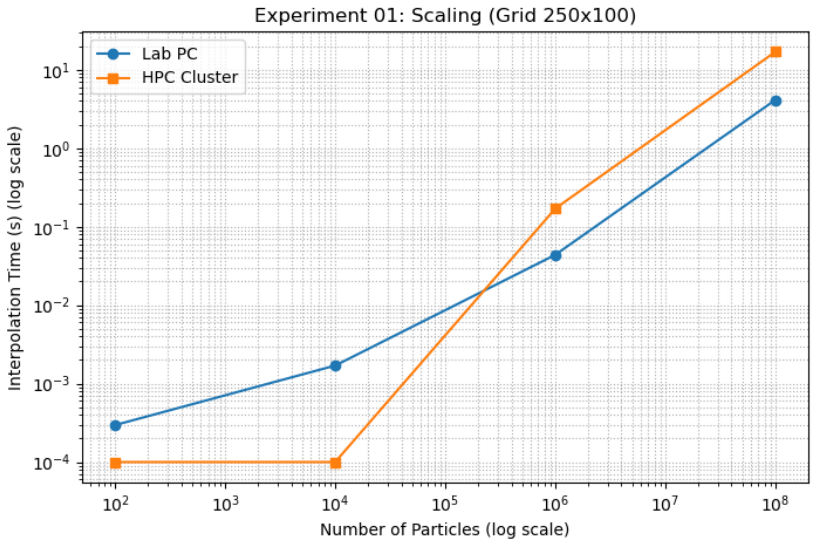

Figure 2: Scaling execution time for Grid 250x100.

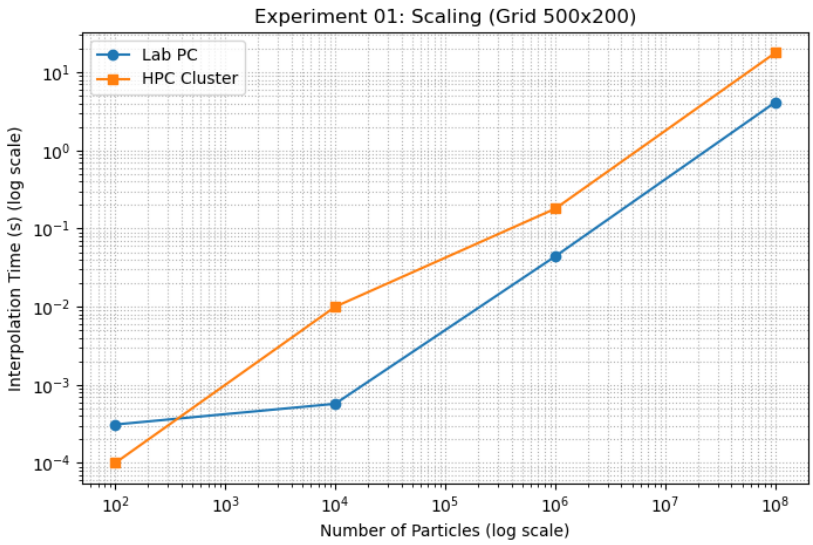

Figure 3: Scaling execution time for Grid 500x200.

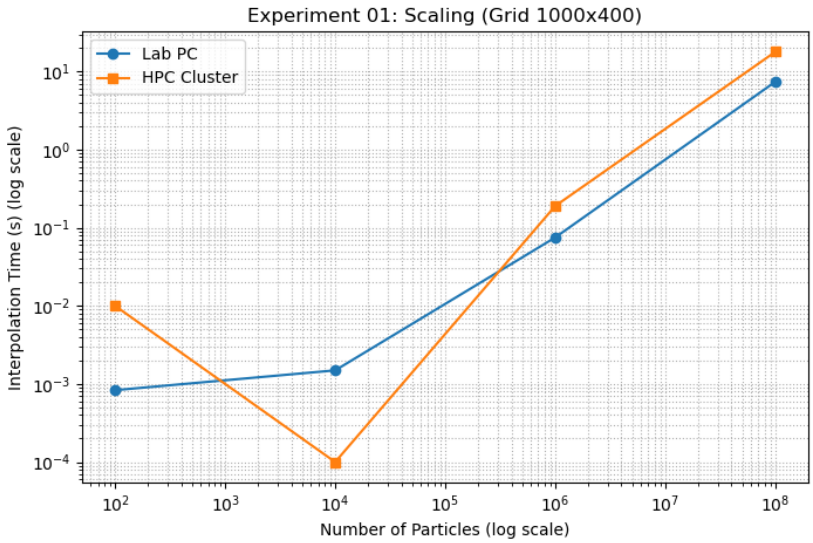

Figure 4: Scaling execution time for Grid 1000x400.

4.2 Experiment 02: Consistency Across Configurations

This experiment evaluates the consistency of interpolation performance across different grid configurations when particle initialization is performed only once. The number of particles is fixed at:

$$10^8$$

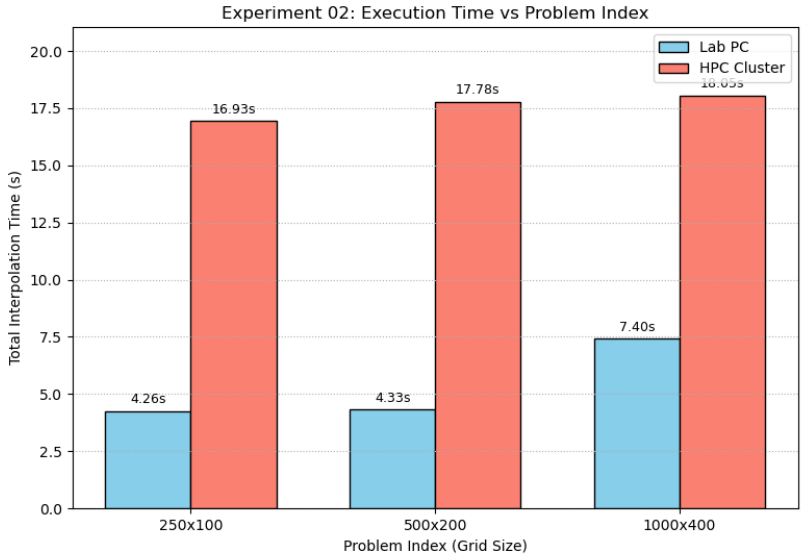

Figure 5: Interpolation Time vs Problem Index for Lab PC and HPC Cluster.

4.3 Experiment 03: The Mover Operation and OpenMP Parallelization

After interpolation, particles are displaced within the domain. If a move attempts to place a particle outside the domain, we regenerate the displacement until the updated position remains inside the domain.

4.3.1 Lab PC Results

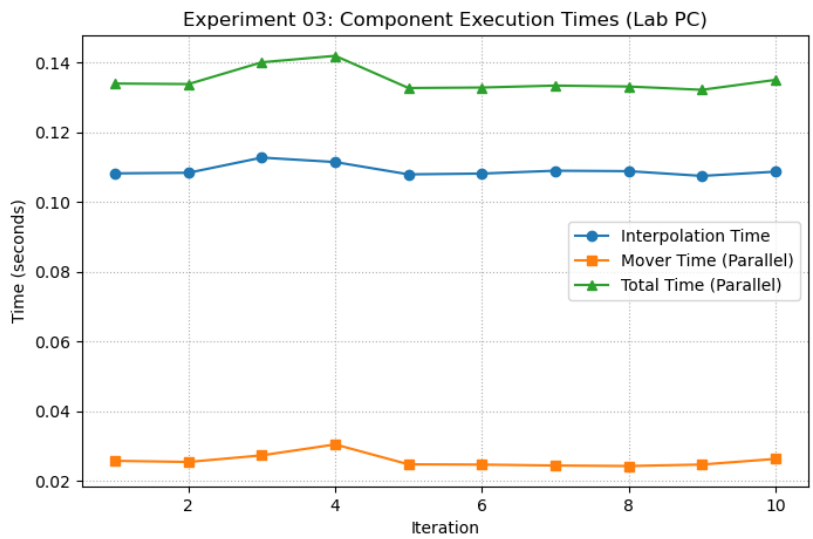

Figure 6: Iteration vs Component Execution Times (Lab PC).

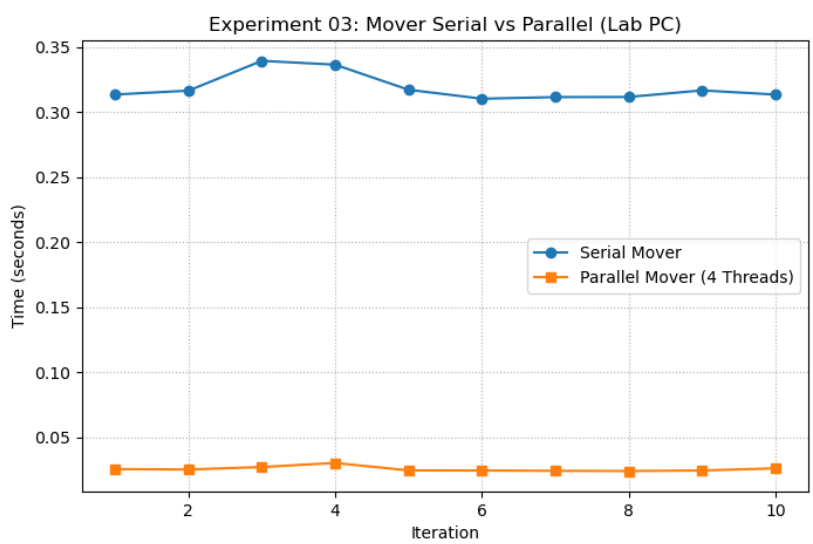

Figure 7: Mover Serial vs Parallel Execution Times (Lab PC).

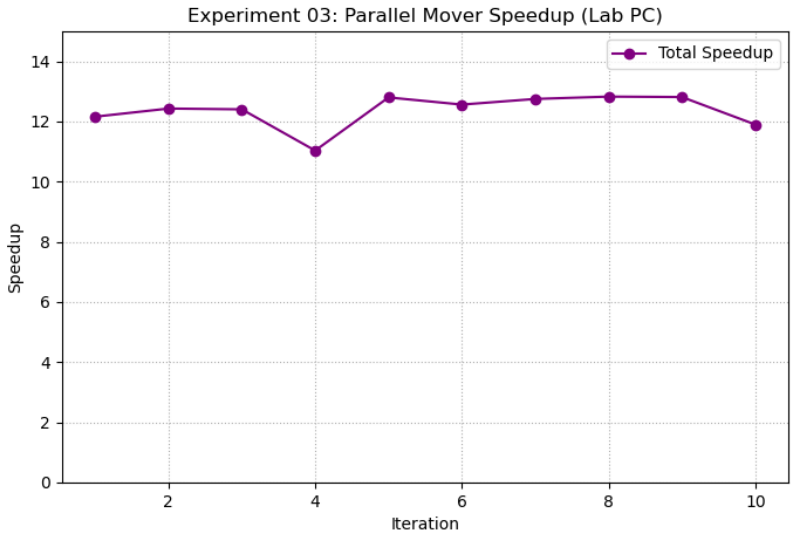

Figure 8: Parallel Mover Speedup (Lab PC).

4.3.2 HPC Cluster Results

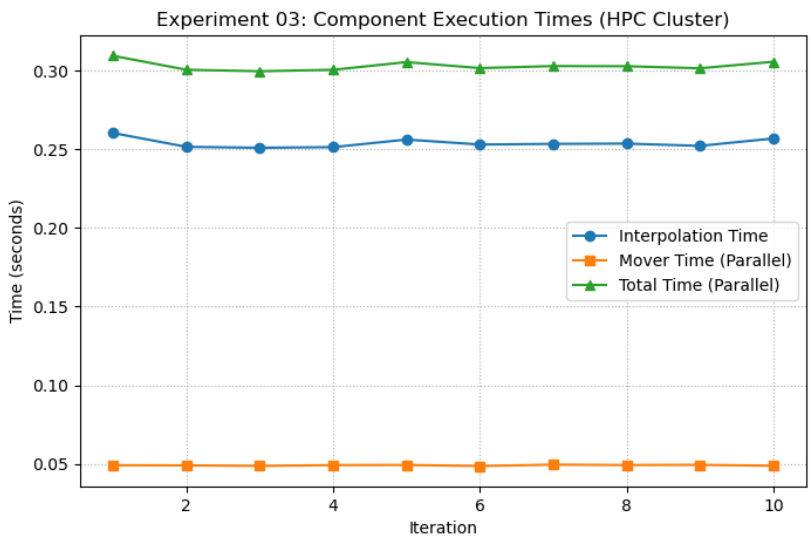

Figure 9: Iteration vs Component Execution Times (HPC Cluster).

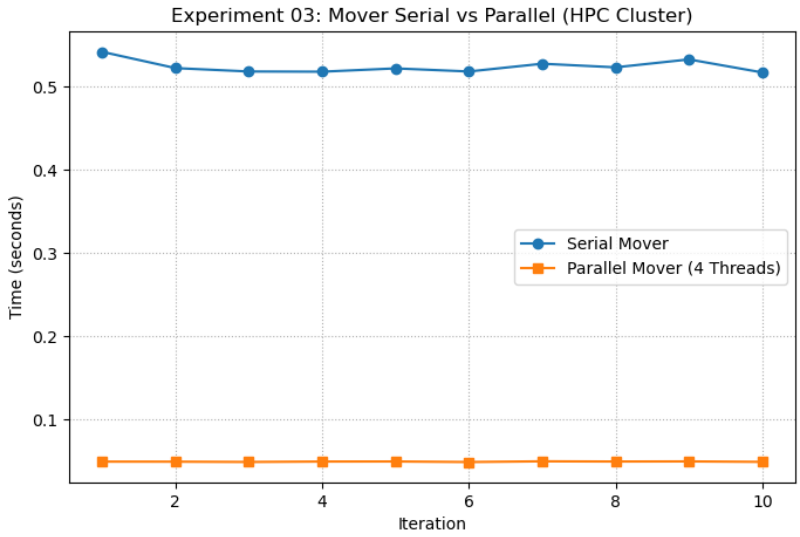

Figure 10: Mover Serial vs Parallel Execution Times (HPC Cluster).

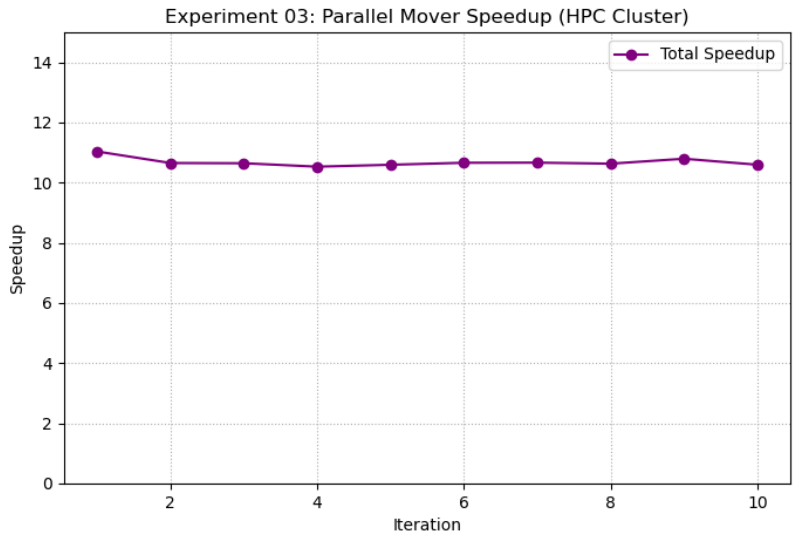

Figure 11: Parallel Mover Speedup (HPC Cluster).

5 Analysis and Questions

5.1 Performance Differences: Lab PC vs. HPC Cluster

The Lab PC (Intel i5-12500) features a higher single-thread clock speed compared to the HPC Cluster (Intel Xeon E5-2640 v3). Because of this frequency advantage, the strictly serial interpolation code executes noticeably faster on the Lab PC.

The HPC Cluster utilizes a dual-socket NUMA architecture. Without careful memory management, memory allocated on one socket and processed on another introduces severe latency penalties. By implementing a NUMA First-Touch policy (initializing memory pages with the thread that will subsequently process them), we ensured stable memory scaling, though the raw serial speed remained lower than the Lab PC due to the clock speed differential.

5.2 Parallel Speedup and Hardware Properties

The maximum theoretical parallel speedup relies on the number of threads utilized, which is fixed:

$$Threads = 4$$

We observed super-linear speedup on both architectures. For the Lab PC, the total wall-clock speedup averaged:

$$S \approx 12.5$$

For the HPC Cluster, the total wall-clock speedup averaged:

$$S \approx 10.6$$

This super-linear scaling is the result of a necessary algorithmic modification. The serial mover relied on the standard random generation function, which maintains a hidden global state and incurs significant instruction overhead. The parallel mover was rewritten to use thread-local random seed states. The new algorithm required approximately three times less raw CPU work than the serial algorithm. When this algorithmic efficiency was multiplied by the perfect linear scaling across the 4 physical cores, the total observed speedup exceeded the hardware thread count.

5.3 Parallelizing the Interpolation Function

Parallelizing the interpolation function is complex due to inherent race conditions. The interpolation logic maps scattered points to specific grid cells.

If multiple OpenMP threads process different particles that happen to be located near each other, those threads will simultaneously attempt to update the exact same shared grid points. This creates a data race, leading to corrupted summation values in the mesh array.

To resolve this bottleneck, one could use atomic addition directives. However, atomic operations serialize memory access at the hardware level, creating massive contention if particles are densely clustered. A more performant approach requires grid privatization, where every thread holds an independent copy of the entire mesh to accumulate partial weights, followed by a final reduction step to sum all private meshes into a single global mesh.